

Course: ENSF 480 – Principles of Software Design

Lab #: Lab 3

Instructor Name: Mahmood Moussavi

Student Name(s): Stephen Ravelo, Aaron Lauang

Lab Section: B02

Date submitted: September 27, 2025

Exercise A

circle.h

```
/*
 * File Name: circle.h
 * Assignment: Lab 3 Exercise A
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#ifndef CIRCLE_H
#define CIRCLE_H

#include "shape.h"

class Circle : virtual public Shape
{
public:
    Circle(double x, double y, double radius, const char *shapeName);
    Circle(const Circle &source);
    Circle &operator=(const Circle &rhs);

    double get_radius() const;
    void set_radius(const double side_b);

    void display() const;
    double area() const;
    double perimeter() const;

private:
    double radiusM;
};

#endif
```

circle.cpp

```
/*
 * File Name: circle.cpp
 * Assignment: Lab 3 Exercise A
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#include <iostream>
#include <iomanip>
#include <math.h>
#include <cstring>

using namespace std;

#include "circle.h"

Circle::Circle(double x, double y, double radius, const char *shapeName)
    : Shape(x, y, shapeName), radiusM(radius)
{
}

Circle::Circle(const Circle &source)
    : Shape(source.getOrigin().getX(), source.getOrigin().getY(),
source.getName()), radiusM(source.radiusM)
{
}

Circle &Circle::operator=(const Circle &rhs)
{
    if (this != &rhs)
    {
        Shape::operator=(rhs);
        set_radius(rhs.radiusM);
    }

    return *this;
}

double Circle::get_radius() const { return radiusM; }
void Circle::set_radius(double radius) { radiusM = radius; }

double Circle::area() const { return M_PI * pow(radiusM, 2); }
double Circle::perimeter() const { return 2 * M_PI * radiusM; }
```

```

void Circle::display() const
{
    cout << fixed << setprecision(2);

    cout << "Shape Name: " << getName() << endl
        << "X-coordinate: " << getOrigin().getX() << endl
        << "Y-coordinate: " << getOrigin().getY() << endl
        << "Radius: " << radiusM << endl
        << "Area: " << area() << endl
        << "Perimeter: " << perimeter() << endl;
}

```

curveCut.h

```

/*
 * File Name: curveCut.h
 * Assignment: Lab 3 Exercise A
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#ifndef CURVECUT_H
#define CURVECUT_H

#include "rectangle.h"
#include "circle.h"

class CurveCut : public Rectangle, public Circle
{
public:
    CurveCut(double x, double y, double side_a, double side_b, double
radius, const char *shapeName);
    CurveCut(const CurveCut &source);
    CurveCut &operator=(const CurveCut &rhs);

    void display() const;
    double area() const;
    double perimeter() const;
};

#endif

```

curveCut.cpp

```
/*
 * File Name: curveCut.cpp
 * Assignment: Lab 3 Exercise A
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#include <iostream>
#include <iomanip>
#include <math.h>
#include <cstring>

using namespace std;

#include "curveCut.h"

CurveCut::CurveCut(double x, double y, double side_a, double side_b,
double radius, const char *shapeName)
    : Shape(x, y, shapeName),
      Rectangle(x, y, side_a, side_b, shapeName),
      Circle(x, y, radius, shapeName)
{
    if (radius > side_a || radius > side_b)
    {
        cout << "Error: CurveCut circle radius is greater than side_a or
side_b." << endl;
        exit(1);
    }
}

CurveCut::CurveCut(const CurveCut &source)
    : Shape(source.getOrigin().getX(), source.getOrigin().getY(),
source.getName()),
      Rectangle(source.getOrigin().getX(), source.getOrigin().getY(),
source.get_side_a(), source.get_side_b(), source.getName()),
      Circle(source.getOrigin().getX(), source.getOrigin().getY(),
source.get_radius(), source.getName())
{
}

CurveCut &CurveCut::operator=(const CurveCut &rhs)
{
    if (this != &rhs)
```

```

    {
        Rectangle::operator=(rhs);
        Circle::operator=(rhs);
    }

    return *this;
}

double CurveCut::area() const
{
    double circle_area = M_PI * pow(get_radius(), 2);
    double rectangle_area = get_side_a() * get_side_b();

    return rectangle_area - circle_area / 4;
}

double CurveCut::perimeter() const
{
    return 2 * (get_side_a() + get_side_b() - get_radius()) + M_PI *
get_radius() / 2;
}

void CurveCut::display() const
{
    cout << fixed << setprecision(2);

    cout << "Shape Name: " << getName() << endl
        << "X-coordinate: " << getOrigin().getX() << endl
        << "Y-coordinate: " << getOrigin().getY() << endl
        << "Width: " << get_side_a() << endl
        << "Length: " << get_side_b() << endl
        << "Radius: " << get_radius() << endl
        << "Area: " << area() << endl
        << "Perimeter: " << perimeter() << endl;
}

```

Output

```
Testing Functions in class Circle:
Shape Name: CIRCLE C
X-coordinate: 3.00
Y-coordinate: 5.00
Radius: 9.00
Area: 254.47
Perimeter: 56.55
the area of CIRCLE C is: 254.47
the perimeter of CIRCLE C is: 56.55

The distance between rectangle a and circle c is: 2.83
Shape Name: CurveCut rc
X-coordinate: 6.00
Y-coordinate: 5.00
Width: 10.00
Length: 12.00
Radius: 9.00
Area: 56.38
Perimeter: 40.14
the area of CurveCut rc is: 56.38
the perimeter of CurveCut rc is: 40.14

The distance between rc and c is: 3.00
Shape Name: SQUARE - S
X-coordinate: 5.00
Y-coordinate: 7.00
Side a: 12.00
Area: 144.00
Perimeter: 48.00

the area of SQUARE - S is: 144.00

the perimeter of SQUARE - S is: 48.00
Shape Name: RECTANGLE A
X-coordinate: 5.00
Y-coordinate: 7.00
Side a: 400.00
Side b: 300.00
Area: 120000.00
Perimeter: 1400.00
```

the area of RECTANGLE A is: 120000.00

the perimeter of SQUARE - S is: 1400.00

Shape Name: CIRCLE C

X-coordinate: 3.00

Y-coordinate: 5.00

Radius: 9.00

Area: 254.47

Perimeter: 56.55

the area of CIRCLE C is: 254.47

the circumference of CIRCLE C is: 56.55

Shape Name: CurveCut rc

X-coordinate: 6.00

Y-coordinate: 5.00

Width: 10.00

Length: 12.00

Radius: 9.00

Area: 56.38

Perimeter: 40.14

the area of CurveCut rc is: 56.38

the perimeter of CurveCut rc is: 40.14

Testing copy constructor in class CurveCut:

Shape Name: CurveCut rc

X-coordinate: 6.00

Y-coordinate: 5.00

Width: 10.00

Length: 12.00

Radius: 9.00

Area: 56.38

Perimeter: 40.14

Testing assignment operator in class CurveCut:

Shape Name: CurveCut cc2

X-coordinate: 2.00

Y-coordinate: 5.00

Width: 100.00

Length: 12.00

Radius: 9.00

Area: 1136.38

Perimeter: 220.14

Shape Name: CurveCut cc2

X-coordinate: 6.00

Y-coordinate: 5.00

Width: 10.00

Length: 12.00

Radius: 9.00

Area: 56.38

Perimeter: 40.14

Exercise B

iterator.cpp

```
/*
 * File Name: iterator.cpp
 * Assignment: Lab 3 Exercise B
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#include <iostream>
#include <assert.h>
#include <string.h>
#include "mystring2.h"

using namespace std;

template <class T>
class Vector
{
public:
    class VectIter
    {
        friend class Vector;

    private:
        Vector *v; // points to a vector object of type T
        int index; // represents the subscript number of the vector's
                  // array.

    public:
        VectIter(Vector &x);

        T operator++();
        // PROMISES: increments the iterator's index and return the
        //              value of the element at the index position. If
        //              index exceeds the size of the array it will
        //              be set to zero. Which means it will be circulated
        //              back to the first element of the vector.

        T operator++(int);
        // PROMISES: returns the value of the element at the index
        //              position, then increments the index. If
        //              index exceeds the size of the array it will
        //              be set to zero. Which means it will be circulated
```

```

        //          back to the first element of the vector.

    T operator--();
    // PROMISES: decrements the iterator index, and return the
    //          the value of the element at the index. If
    //          index is less than zero it will be set to the
    //          last element in the array. Which means it will be
    //          circulated to the last element of the vector.

    T operator--(int);
    // PROMISES: returns the value of the element at the index
    //          position, then decrements the index. If
    //          index is less than zero it will be set to the
    //          last element in the array. Which means it will be
    //          circulated to the last element of the vector.

    T operator*();
    // PROMISES: returns the value of the element at the current
    //          index position.
};

Vector(int sz);
~Vector();

T &operator[](int i);
// PROMISES: returns existing value in the ith element of
//          array or sets a new value to the ith element in
//          array.

void ascending_sort();
// PROMISES: sorts the vector values in ascending order.

private:
    T *array;          // points to the first element of an array of T
    int size;          // size of array
    void swap(T &, T &); // swaps the values of two elements in array
public:
};

template <class T>
void Vector<T>::ascending_sort()
{
    for (int i = 0; i < size - 1; i++)
        for (int j = i + 1; j < size; j++)
            if (array[i] > array[j])

```

```

        swap(array[i], array[j]);
    }

template <>
void Vector<const char *>::ascending_sort()
{
    for (int i = 0; i < size - 1; i++)
        for (int j = i + 1; j < size; j++)
            if (strcmp(array[i], array[j]) > 0)
                swap(array[i], array[j]);
}

```

```

template <class T>
void Vector<T>::swap(T &a, T &b)
{
    T tmp = a;
    a = b;
    b = tmp;
}

```

```

template <class T>
T Vector<T>::VectIter::operator++()
{
    if (++index >= v->size)
        index = 0;
    T tmp = v->array[index];
    return tmp;
}

```

```

template <class T>
T Vector<T>::VectIter::operator++(int)
{
    T tmp = v->array[index++];
    if (index >= v->size)
        index = 0;
    return tmp;
}

```

```

template <class T>
T Vector<T>::VectIter::operator--()
{
    if (--index < 0)
        index = v->size - 1;
    T tmp = v->array[index];
    return tmp;
}

```

```

}

template <class T>
T Vector<T>::VectIter::operator--(int)
{
    T tmp = v->array[index--];
    if (index < 0)
        index = v->size - 1;
    return tmp;
}

template <class T>
T Vector<T>::VectIter::operator*()
{
    return v->array[index];
}

template <class T>
Vector<T>::VectIter::VectIter(Vector &x)
{
    v = &x;
    index = 0;
}

template <class T>
Vector<T>::Vector(int sz)
{
    size = sz;
    array = new T[sz];
    assert(array != NULL);
}

template <class T>
Vector<T>::~~Vector()
{
    delete[] array;
    array = NULL;
}

template <class T>
T &Vector<T>::operator[](int i)
{
    return array[i];
}

```

```

int main()
{

    Vector<int> x(3);
    x[0] = 999;
    x[1] = -77;
    x[2] = 88;

    Vector<int>::VectIter iter(x);

    cout << "\n\nThe first element of vector x contains: " << *iter;

    // the code between the #if 0 and #endif is ignored by
    // compiler. If you change it to #if 1, it will be compiled
#if 1
    cout << "\n\nTesting an <int> Vector: " << endl;

    cout << "\n\nTesting sort";
    x.ascending_sort();

    for (int i = 0; i < 3; i++)
        cout << endl
            << iter++;

    cout << "\n\nTesting Prefix --:";
    for (int i = 0; i < 3; i++)
        cout << endl
            << --iter;

    cout << "\n\nTesting Prefix ++:";
    for (int i = 0; i < 3; i++)
        cout << endl
            << ++iter;

    cout << "\n\nTesting Postfix --:";
    for (int i = 0; i < 3; i++)
        cout << endl
            << iter--;

    cout << endl;

    cout << "Testing a <Mystring> Vector: " << endl;
    Vector<Mystring> y(3);
    y[0] = "Bar";

```

```

y[1] = "Foo";
y[2] = "All";
;

Vector<Mystring>::VectIter iters(y);

cout << "\n\nTesting sort";
y.ascending_sort();

for (int i = 0; i < 3; i++)
    cout << endl
        << iters++;

cout << "\n\nTesting Prefix --:";
for (int i = 0; i < 3; i++)
    cout << endl
        << --iters;

cout << "\n\nTesting Prefix ++:";
for (int i = 0; i < 3; i++)
    cout << endl
        << ++iters;

cout << "\n\nTesting Postfix --:";
for (int i = 0; i < 3; i++)
    cout << endl
        << iters--;

cout << endl;
cout << "Testing a <char *> Vector: " << endl;
Vector<const char *> z(3);
z[0] = "Orange";
z[1] = "Pear";
z[2] = "Apple";
;

Vector<const char *>::VectIter iterchar(z);

cout << "\n\nTesting sort";
z.ascending_sort();

for (int i = 0; i < 3; i++)
    cout << endl
        << iterchar++;

```

```

#endif
cout << "\nPrgram Terminated Successfully." << endl;

return 0;
}

```

mystring2.h

```

/*
 * File Name: mystring2.h
 * Assignment: Lab 3 Exercise B
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#ifndef MYSTRING_H
#define MYSTRING_H
#include <iostream>
class Mystring {
    friend std::ostream& operator <<(std::ostream& os, const Mystring& s);
public:
    Mystring();
    // PROMISES: Empty string object is created.

    Mystring(int n);
    // PROMISES: Creates an empty string with a total capacity of n.
    //           In other words, dynamically allocates n elements for
    //           charsM, sets the lengthM to zero, and fills the first
    //           element of charsM with '\0'.

    Mystring(const char *s);
    // REQUIRES: s points to first char of a built-in string.
    // REQUIRES: Mystring object is created by copying chars from s.

    ~Mystring(); // destructor

    Mystring(const Mystring& source); // copy constructor

    Mystring& operator =(const Mystring& rhs); // assignment operator
    // REQUIRES: rhs is reference to a Mystring as a source
    // PROMISES: to make this-object (object that this is pointing to, as a
copy
    //           of rhs.

```

```

bool operator>(const Mystring &rhs);

int length() const;
// PROMISES: Return value is number of chars in charsM.

char get_char(int pos) const;
// REQUIRES: pos >= 0 && pos < length()
// PROMISES:
// Return value is char at position pos.
// (The first char in the charsM is at position 0.)

const char * c_str() const;
// PROMISES:
// Return value points to first char in built-in string
// containing the chars of the string object.

void set_char(int pos, char c);
// REQUIRES: pos >= 0 && pos < length(), c != '\0'
// PROMISES: Character at position pos is set equal to c.

Mystring& append(const Mystring& other);

// PROMISES: extends the size of charsM to allow concatenate
other.charsM to
// to the end of charsM. For example if charsM points to
"ABC", and
// other.charsM points to XYZ, extends charsM to "ABCXYZ".
//

void set_str(char* s);
// REQUIRES: s is a valid C++ string of characters (a built-in string)
// PROMISES: copies s into charsM, if the length of s is less than or
equal lengthM.
// Otherwise, extends the size of the charsM to s.lengthM+1,
and copies
// s into the charsM.

private:

int lengthM; // the string length - number of characters excluding \0
char* charsM; // a pointer to the beginning of an array of characters,
allocated dynamically.
void memory_check(char* s);
// PROMISES: if s points to NULL terminates the program.
};

```



```
#endif
```

mystring2.cpp

```
/*
 * File Name: mystring2.cpp
 * Assignment: Lab 3 Exercise B
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#include "mystring2.h"
#include <string.h>
#include <iostream>
using namespace std;

Mystring::Mystring()
{
    charsM = new char[1];
    charsM[0] = '\0';
    lengthM = 0;
}

Mystring::Mystring(const char *s)
    : lengthM((int)strlen(s))
{
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}

Mystring::Mystring(int n)
    : lengthM(0), charsM(new char[n])
{
    charsM[0] = '\0';
}

Mystring::Mystring(const Mystring &source) : lengthM(source.lengthM),
charsM(new char[source.lengthM + 1])
{
    strcpy(charsM, source.charsM);
}

Mystring::~Mystring()
{
}
```

```

    delete[] charsM;
}

int Mystring::length() const
{
    return lengthM;
}

char Mystring::get_char(int pos) const
{
    if (pos < 0 && pos >= length())
    {
        cerr << "\nERROR: get_char: the position is out of boundary.";
    }

    return charsM[pos];
}

const char *Mystring::c_str() const
{
    return charsM;
}

void Mystring::set_char(int pos, char c)
{
    if (pos < 0 && pos >= length())
    {
        cerr << "\nset_char: the position is out of boundary."
              << " Nothing was changed.";
        return;
    }

    if (c != '\0')
    {
        cerr << "\nset_char: char c is empty."
              << " Nothing was changed.";
        return;
    }

    charsM[pos] = c;
}

Mystring &Mystring::operator=(const Mystring &S)
{
    if (this == &S)

```

```

        return *this;
    delete[] charsM;
    lengthM = (int)strlen(S.charsM);
    charsM = new char[lengthM + 1];
    strcpy(charsM, S.charsM);
    return *this;
}

bool Mystring::operator>(const Mystring &rhs)
{
    return strcmp(charsM, rhs.charsM) > 0;
}

Mystring &Mystring::append(const Mystring &other)
{
    char *tmp = new char[lengthM + other.lengthM + 1];
    lengthM += other.lengthM;
    strcpy(tmp, charsM);
    strcat(tmp, other.charsM);
    delete[] charsM;
    charsM = tmp;

    return *this;
}

void Mystring::set_str(char *s)
{
    delete[] charsM;
    lengthM = (int)strlen(s);
    charsM = new char[lengthM + 1];

    strcpy(charsM, s);
}

std::ostream &operator<<(std::ostream &os, const Mystring &s)
{
    return os << s.charsM;
}

```

Output

```
The first element of vector x contains: 999
Testing an <int> Vector:

Testing sort
-77
88
999

Testing Prefix --:
999
88
-77

Testing Prefix ++:
88
999
-77

Testing Postfix --
-77
999
88

Testing a <Mystring> Vector:

Testing sort
All
Bar
Foo

Testing Prefix --:
Foo
Bar
All

Testing Prefix ++:
Bar
Foo
All

Testing Postfix --
All
Foo
Bar
Testing a <char *> Vector:

Testing sort
Apple
Orange
Pear
Program Terminated Successfully.
```

Exercise C

lookupTable.h

```
/*
 * File Name: LookupTable.h
 * Assignment: Lab 3 Exercise C
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#ifndef LOOKUPTABLE_H
#define LOOKUPTABLE_H
#include <iostream>
using namespace std;

// class LookupTable: GENERAL CONCEPTS
//
//   key/datum pairs are ordered.  The first pair is the pair with
//   the lowest key, the second pair is the pair with the second
//   lowest key, and so on.  This implies that you must be able to
//   compare two keys with the < operator.
//
//   Each LookupTable has an embedded iterator class that allows users
//   of the class to traverse through the list and have access to each
//   node.

#include "customer.h"

//   In this version of the LookupTable a new struct type called Pair
//   is introduced which represents a key/data pair.

// typedef int LT_Key;
// typedef Customer LT_Datum;

template <class K, class D>
class LookupTable;

template <class K, class D>
ostream &operator<<(ostream &os, const LookupTable<K, D> &lt);

template <class K, class D>
struct Pair;

template <class K, class D>
```

```

struct Pair
{
    Pair(K keyA, D datumA) : key(keyA), datum(datumA)
    {
    }

    K key;
    D datum;
};

template <class K, class D>
class LT_Node
{
    friend class LookupTable<K, D>;

private:
    Pair<K, D> pairM;
    LT_Node *nextM;

    // This ctor should be convenient in insert and copy operations.
    LT_Node(const Pair<K, D> &pairA, LT_Node *nextA);
};

template <class K, class D>
class LookupTable
{
public:
    // Nested class
    class Iterator
    {
        friend class LookupTable;
        LookupTable *LT;
        // LT_Node* cursor;

    public:
        Iterator() : LT(0) {}
        Iterator(LookupTable &x) : LT(&x) {}
        const D &operator*();
        const D &operator++();
        const D &operator++(int);
        int operator!();

        void step_fwd()
        {
            assert(LT->cursor_ok());

```

```

        LT->step_fwd();
    }
};

LookupTable();
LookupTable(const LookupTable &source);
LookupTable &operator=(const LookupTable &rhs);
~LookupTable();

LookupTable &begin();

int size() const;
// PROMISES: Returns number of keys in the table.

int cursor_ok() const;
// PROMISES:
// Returns 1 if the cursor is attached to a key/datum pair,
// and 0 if the cursor is in the off-list state.

const K &cursor_key() const;
// REQUIRES: cursor_ok()
// PROMISES: Returns key of key/datum pair to which cursor is attached.

const D &cursor_datum() const;
// REQUIRES: cursor_ok()
// PROMISES: Returns datum of key/datum pair to which cursor is
attached.

void insert(const Pair<K, D> &pairA);
// PROMISES:
// If keyA matches a key in the table, the datum for that
// key is set equal to datumA.
// If keyA does not match an existing key, keyA and datumM are
// used to create a new key/datum pair in the table.
// In either case, the cursor goes to the off-list state.

void remove(const K &keyA);
// PROMISES:
// If keyA matches a key in the table, the corresponding
// key/datum pair is removed from the table.
// If keyA does not match an existing key, the table is unchanged.
// In either case, the cursor goes to the off-list state.

void find(const K &keyA);
// PROMISES:

```

```

// If keyA matches a key in the table, the cursor is attached
// to the corresponding key/datum pair.
// If keyA does not match an existing key, the cursor is put in
// the off-list state.

void go_to_first();
// PROMISES: If size() > 0, cursor is moved to the first key/datum pair
// in the table.

void step_fwd();
// REQUIRES: cursor_ok()
// PROMISES:
// If cursor is at the last key/datum pair in the list, cursor
// goes to the off-list state.
// Otherwise the cursor moves forward from one pair to the next.

void make_empty();
// PROMISES: size() == 0.

template <class K2, class D2>
friend ostream &operator<<(ostream &os, const LookupTable<K2, D2> &lt);

private:
    int sizeM;
    LT_Node<K, D> *headM;
    LT_Node<K, D> *cursorM;

    void destroy();
    // Deallocate all nodes, set headM to zero.

    void copy(const LookupTable &source);
    // Establishes *this as a copy of source. Cursor of *this will
    // point to the twin of whatever the source's cursor points to.
};

#endif

template <class K, class D>
LookupTable<K, D> &LookupTable<K, D>::begin()
{
    cursorM = headM;
    return *this;
}

template <class K, class D>

```



```

LT_Node<K, D>::LT_Node(const Pair<K, D> &pairA, LT_Node<K, D> *nextA)
    : pairM(pairA), nextM(nextA)
{
}

template <class K, class D>
LookupTable<K, D>::LookupTable()
    : sizeM(0), headM(0), cursorM(0)
{
}

template <class K, class D>
LookupTable<K, D>::LookupTable(const LookupTable<K, D> &source)
{
    copy(source);
}

template <class K, class D>
LookupTable<K, D> &LookupTable<K, D>::operator=(const LookupTable<K, D>
&rhs)
{
    if (this != &rhs)
    {
        destroy();
        copy(rhs);
    }
    return *this;
}

template <class K, class D>
LookupTable<K, D>::~~LookupTable()
{
    destroy();
}

template <class K, class D>
int LookupTable<K, D>::size() const
{
    return sizeM;
}

template <class K, class D>
int LookupTable<K, D>::cursor_ok() const
{
    return cursorM != 0;
}

```

```

}

template <class K, class D>
const K &LookupTable<K, D>::cursor_key() const
{
    assert(cursor_ok());
    return cursorM->pairM.key;
}

template <class K, class D>
const D &LookupTable<K, D>::cursor_datum() const
{
    assert(cursor_ok());
    return cursorM->pairM.datum;
}

template <class K, class D>
void LookupTable<K, D>::insert(const Pair<K, D> &pairA)
{
    // Add new node at head?
    if (headM == 0 || pairA.key < headM->pairM.key)
    {
        headM = new LT_Node<K, D>(pairA, headM);
        sizeM++;
    }

    // Overwrite datum at head?
    else if (pairA.key == headM->pairM.key)
        headM->pairM.datum = pairA.datum;

    // Have to search ...

    else
    {
        LT_Node<K, D> *before = headM;
        LT_Node<K, D> *after = headM->nextM;

        while (after != NULL && (pairA.key > after->pairM.key))
        {
            before = after;
            after = after->nextM;
        }

        if (after != NULL && pairA.key == after->pairM.key)
        {

```

```

        after->pairM.datum = pairA.datum;
    }
    else
    {
        before->nextM = new LT_Node<K, D>(pairA, before->nextM);
        sizeM++;
    }
}
}

template <class K, class D>
void LookupTable<K, D>::remove(const K &keyA)
{
    if (headM == 0 || keyA < headM->pairM.key)
        return;

    LT_Node<K, D> *doomed_node = 0;
    if (keyA == headM->pairM.key)
    {
        doomed_node = headM;
        headM = headM->nextM;
        sizeM--;
    }
    else
    {
        LT_Node<K, D> *before = headM;
        LT_Node<K, D> *maybe_doomed = headM->nextM;
        while (maybe_doomed != 0 && keyA > maybe_doomed->pairM.key)
        {
            before = maybe_doomed;
            maybe_doomed = maybe_doomed->nextM;
        }

        if (maybe_doomed != 0 && maybe_doomed->pairM.key == keyA)
        {
            doomed_node = maybe_doomed;
            before->nextM = maybe_doomed->nextM;
            sizeM--;
        }
    }
    delete doomed_node; // Does nothing if doomed_node == 0.
}

template <class K, class D>

```

```

void LookupTable<K, D>::find(const K &keyA)
{
    LT_Node<K, D> *ptr = headM;
    while (ptr != NULL && ptr->pairM.key != keyA)
    {
        ptr = ptr->nextM;
    }

    cursorM = ptr;
}

```

```

template <class K, class D>
void LookupTable<K, D>::go_to_first()
{
    cursorM = headM;
}

```

```

template <class K, class D>
void LookupTable<K, D>::step_fwd()
{
    assert(cursor_ok());
    cursorM = cursorM->nextM;
}

```

```

template <class K, class D>
void LookupTable<K, D>::make_empty()
{
    destroy();
    sizeM = 0;
    cursorM = 0;
}

```

```

template <class K, class D>
void LookupTable<K, D>::destroy()
{
    LT_Node<K, D> *ptr = headM;
    while (ptr != NULL)
    {
        headM = headM->nextM;
        delete ptr;
        ptr = headM;
    }
    cursorM = NULL;
    sizeM = 0;
}

```

```

}

template <class K, class D>
void LookupTable<K, D>::copy(const LookupTable &source)
{
    headM = 0;
    cursorM = 0;

    if (source.headM == 0)
        return;

    for (LT_Node<K, D> *p = source.headM; p != 0; p = p->nextM)
    {
        insert(Pair(p->pairM.key, p->pairM.datum));
        if (source.cursorM == p)
            find(p->pairM.key);
    }
}

template <class K, class D>
ostream &operator<<(ostream &os, const LookupTable<K, D> &lt)
{
    if (lt.cursor_ok())
        os << lt.cursor_key() << " " << lt.cursor_datum();
    else
        os << "Not Found.";

    return os;
}

template <class K, class D>
const D &LookupTable<K, D>::Iterator::operator*()
{
    assert(LT->cursor_ok());
    return LT->cursor_datum();
}

template <class K, class D>
const D &LookupTable<K, D>::Iterator::operator++()
{
    assert(LT->cursor_ok());
    const D &x = LT->cursor_datum();
    LT->step_fwd();
    return x;
}

```

```
}

template <class K, class D>
const D &LookupTable<K, D>::Iterator::operator++(int)
{
    assert(LT->cursor_ok());

    LT->step_fwd();
    return LT->cursor_datum();
}

template <class K, class D>
int LookupTable<K, D>::Iterator::operator!()
{
    return (LT->cursor_ok());
}
```

mainLab3ExC.cpp

```
/*
 * File Name: mainLab3ExC.cpp
 * Assignment: Lab 3 Exercise C
 * Lab Section: B02
 * Completed by: Stephen Ravelo, Aaron Lauang
 * Submission Date: September 28, 2025
 */

#include <assert.h>
#include <iostream>
#include "lookupTable.h"
#include "customer.h"
#include <cstring>
using namespace std;

template <class K, class D>
void print(LookupTable<K, D> &lt);

template <class K, class D>
void try_to_find(LookupTable<K, D> &lt, K key);

void test_Customer();

// Uncomment the following function calls when ready to test template
// class LookupTable
void test_String();
void test_integer();

int main()
{
    // create and test a lookup table with an integer key value and Customer
    // datum
    test_Customer();

    // Uncomment the following function calls when ready to test template
    // class LookupTable
    // create and test a a lookup table of type <int, String>
    test_String();

    // Uncomment the following function calls when ready to test template
    // class LookupTable
    // create and test a a lookup table of type <int, int>
    test_integer();
}
```

```

    cout << "\n\nProgram terminated successfully.\n\n";

    return 0;
}

template <class K, class D>
void print(LookupTable<K, D> &lt;
{
    if (<.size() == 0)
        cout << "  Table is EMPTY.\n";
    for (<.go_to_first(); <.cursor_ok(); <.step_fwd())
    {
        cout << < << endl;
    }
}

template <class K, class D>
void try_to_find(LookupTable<K, D> &lt;, K key)
{
    <.find(key);
    if (<.cursor_ok())
        cout << "\nFound key:" << <;
    else
        cout << "\nSorry, I couldn't find key: " << key << " in the table.\n";
}

void test_Customer()
// creating a lookup table for customer objects.
{
    cout << "\nCreating and testing Customers Lookup Table <not template>-
... \n";
    LookupTable<int, Customer> lt;

    // Insert using new keys.
    Customer a("Joe", "Morrison", "11 St. Calgary.", "(403)-1111-123333");
    Customer b("Jack", "Lewis", "12 St. Calgary.", "(403)-1111-123334");
    Customer c("Tim", "Hardy", "13 St. Calgary.", "(403)-1111-123335");
    lt.insert(Pair<int, Customer>(8002, a));
    lt.insert(Pair<int, Customer>(8004, c));
    lt.insert(Pair<int, Customer>(8001, b));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
}

```



```

    cout << "\nPrinting table after inserting 3 new keys and 1
removal...\n";
    print(lt);

    // Pretend that a user is trying to look up customers info.

    cout << "\nLet's look up some names ...\n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    // test Iterator
    cout << "\nTesting and using iterator ...\n";
    LookupTable<int, Customer>::Iterator it = lt.begin();
    cout << "\nThe first node contains: " << *it << endl;

    while (!it)
    {
        cout << ++it << endl;
    }

    // test copying
    lt.go_to_first();
    lt.step_fwd();
    LookupTable clt(lt);
    assert(strcmp(clt.cursor_datum().getFname(), "Joe") == 0);

    cout << "\nTest copying: keys should be 8001, and 8002\n";
    print(clt);
    lt.remove(8002);

    // Assignment operator check.
    clt = lt;

    cout << "\nTest assignment operator: key should be 8001\n";
    print(clt);

    // Wipe out the entries in the table.
    lt.make_empty();
    cout << "\nPrinting table for the last time: Table should be
empty...\n";
    print(lt);

    cout << "***----Finished tests on Customers Lookup Table <not template>-
----***\n";
    cout << "PRESS RETURN TO CONTINUE.";

```

```

    cin.get();
}

void test_String()

// creating Lookuptable for Mystring objects
{
    cout << "\nCreating and testing LookupTable <int, Mystring> ..... \n";
    LookupTable<int, Mystring> lt;

    // Insert using new keys.

    Mystring a("I am an ENEL-409 student.");
    Mystring b("C++ is a powerful language for engineers but it's not
easy.");
    Mystring c("Winter 2004");

    lt.insert(Pair<int, Mystring>(8002, a));
    lt.insert(Pair<int, Mystring>(8001, b));
    lt.insert(Pair<int, Mystring>(8004, c));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
    cout << "\nPrinting table after inserting 3 new keys and 1
removal... \n";
    print(lt);

    // Pretend that a user is trying to look up customers info.

    cout << "\nLet's look up some names ... \n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);
    // test Iterator
    LookupTable<int, Mystring>::Iterator it = lt.begin();
    cout << "\nThe first node contains: " << *it << endl;

    while (!it)
    {
        cout << ++it << endl;
    }

    // test copying
    lt.go_to_first();
    lt.step_fwd();
}

```

```

LookupTable<int, Mystring> clt(lt);
assert(strcmp(clt.cursor_datum().c_str(), "I am an ENEL-409 student.")
== 0);

cout << "\nTest copying: keys should be 8001, and 8002\n";
print(clt);
lt.remove(8002);

// Assignment operator check.
clt = lt;

cout << "\nTest assignment operator: key should be 8001\n";
print(clt);

// Wipe out the entries in the table.
lt.make_empty();
cout << "\nPrinting table for the last time: Table should be empty
...\n";
print(lt);

cout << "***----Finished Lab 4 tests on <int> <Mystring>-----***\n";
cout << "PRESS RETURN TO CONTINUE.";
cin.get();
}

void test_integer()

// creating look table of integers

{
    cout << "\nCreating and testing LookupTable <int, int> ..... \n";
    LookupTable<int, int> lt;

    // Insert using new keys.
    lt.insert(Pair<int, int>(8002, 9999));
    lt.insert(Pair<int, int>(8001, 8888));
    lt.insert(Pair<int, int>(8004, 8888));
    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
    cout << "\nPrinting table after inserting 3 new keys and 1
removal...\n";
    print(lt);

    // Pretend that a user is trying to look up customers info.

```

```

cout << "\nLet's look up some names ...\n";
try_to_find(lt, 8001);
try_to_find(lt, 8000);

// test Iterator
LookupTable<int, int>::Iterator it = lt.begin();

while (!it)
{
    cout << ++it << endl;
}

// test copying
lt.go_to_first();
lt.step_fwd();
LookupTable<int, int> clt(lt);
assert(clt.cursor_datum() == 9999);

cout << "\nTest copying: keys should be 8001, and 8002\n";
print(clt);
lt.remove(8002);

// Assignment operator check.
clt = lt;

cout << "\nTest assignment operator: key should be 8001\n";
print(clt);

// Wipe out the entries in the table.
lt.make_empty();
cout << "\nPrinting table for the last time: Table should be empty
...\n";
print(lt);

cout << "***-----Finished Lab 4 tests on <int> <int>-----***\n";
}

```

Output

```
Creating and testing Customers Lookup Table <not template>-...

Printing table after inserting 3 new keys and 1 removal...
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
8002 Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Let's look up some names ...

Found key:8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Sorry, I couldn't find key: 8000 in the table.

Tesing and using iterator ...

The first node contains: Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Test copying: keys should be 8001, and 8002
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
8002 Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Test assignment operator: key should be 8001
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334

Printing table for the last time: Table should be empty...
Table is EMPTY.
***----Finished tests on Customers Lookup Table <not template>----***
PRESS RETURN TO CONTINUE.█
```

Creating and testing LookupTable <int, Mystring>

Printing table after inserting 3 new keys and 1 removal...

8001 C++ is a powerful language for engineers but it's not easy.

8002 I am an ENEL-409 student.

Let's look up some names ...

Found key:8001 C++ is a powerful language for engineers but it's not easy.

Sorry, I couldn't find key: 8000 in the table.

The first node contains: C++ is a powerful language for engineers but it's not easy.

C++ is a powerful language for engineers but it's not easy.

I am an ENEL-409 student.

Test copying: keys should be 8001, and 8002

8001 C++ is a powerful language for engineers but it's not easy.

8002 I am an ENEL-409 student.

Test assignment operator: key should be 8001

8001 C++ is a powerful language for engineers but it's not easy.

Printing table for the last time: Table should be empty ...

Table is EMPTY.

----Finished Lab 4 tests on <int> <Mystring>-----

PRESS RETURN TO CONTINUE.

```
Creating and testing LookupTable <int, int> .....

Printing table after inserting 3 new keys and 1 removal...
8001  8888
8002  9999

Let's look up some names ...

Found key:8001  8888
Sorry, I couldn't find key: 8000 in the table.
8888
9999

Test copying: keys should be 8001, and 8002
8001  8888
8002  9999

Test assignment operator: key should be 8001
8001  8888

Printing table for the last time: Table should be empty ...
Table is EMPTY.
***----Finished Lab 4 tests on <int> <int>-----***

Program terminated successfully.
```